The Open-Source Audit

A Capstone Project Report

Chosen Software: Git — Distributed Version Control System (GPL v2)

Name- Tushar Chakraborty

Register No- 24BAI10842

Course- CSE0002 - OPEN-SOURCE SOFTWARE (LINUX ADMINISTRATION)

Slot- E22

Date of Submission- 30/03/2026

Abstract

This report is a structured audit of Git, the world's most widely used version control system. It covers the origin story behind Git's creation, what the GPL v2 license means for users and companies, the ethical questions that open-source raises, how Git is installed and managed on script uses. a Linux system, the ecosystem it has enabled, and an honest comparison against Perforce Helix Core, its closest proprietary alternative. Five shell scripts demonstrating practical Linux skills are included with full source code, screenshot placeholders, and explanations of the concepts each

1. Origin and Philosophy

1.1 The Problem Git Was Created to Solve

Before Git, teams mostly used CVS or Subversion. Both needed one central server. If the server was slow or down, nobody could work. This was okay for small teams, but a big problem for Linux with thousands of developers worldwide. In 2002 the Linux team started using Bit Keeper (free from Bit Mover). It was good, but not an open source. In 2005 a developer tried to study how it worked. Bit Mover got angry, called it a license violation, and stopped accessing everyone the next day. Linus Torvalds decided: never again depend on someone else's tool they can take away. Instead of asking or buying something else, he wrote to Git himself. In about 10 days, he had a working version. Git was designed to be: • very fast • fully distributed (everyone has a full copy) cryptographically secure

(history can't be secretly changed). Linus later said his rule was simple: "do the opposite of CVS". Within one year, the biggest open-source projects switched to Git. Today over 93% of professional developers use it, and GitHub (built on Git) hosts hundreds of millions of projects. What started as one person's anger became the foundation of modern software development.

1.2 The License — What GPL v2 Actually Says

The Git source code lives at https://github.com/git/git. At the root of the repository is a file called COPYING containing the full text of the GNU General Public License Version 2 (GPL v2) — the same license Torvalds chose for the Linux kernel. Reading this file is not optional; it tells you exactly what you are and is not allowed to do it. Richard Stallman defined four freedoms that make software genuinely free:

1. Freedom 0: Run the software for any purpose without paying or asking permission.

2. Freedom 1: Study how it works and modifies it. This requires access to the source code.

3. Freedom 2: Redistribute copies freely so others can benefit

4. Freedom 3: Distribute your modified versions. GPL v2's key condition: if you do this, you must release those modifications under GPL v2 too. You cannot take the code, improve it, or sell a closed-source version.

Git grants all four freedoms. The critical clause is the copyleft requirement in Freedom 3. This is the core difference between GPL and a

permissive license like MIT. MIT says, do whatever you want, including closing the source. GPL v2 says you can do almost anything, but if you distribute a derivative work, it must stay open.

Can a company modify Git and sell it without sharing changes? Not if they distribute it. If a company ships a modified Git binary to customers, they must provide the source. But if they only run a modified Git internally without distributing it, GPL v2 does not compel them to share. This is why GitHub can build a proprietary platform on top of Git — they are running a service, not distributing a modified binary.

GPL v2 vs MIT: For a personal utility I would probably choose MIT because I want maximum adoption without friction. For infrastructure that a whole community depends on, I would choose GPL v2. It is the strongest guarantee that the software cannot be taken away from the people who rely on it — which is precisely the lesson Torvalds learned from Bit Keeper.

Free as in beer vs free as in freedom: "Free beer" means zero cost. "Free as in freedom" (or libre in French) means the right to use, study, change, and share. Bit Keeper was free as in beer but not free as in freedom — and that distinction cost the entire Linux kernel team for their workflow. Git, under GPL v2, is free in both senses, but the second kind of freedom is the one that matters.

1.3 The Ethics of Open Source

Should all software be open source?

Not necessarily. Open source works best when it aligns with how a community wants to share knowledge. The case is strong: inspectable code is more secure, more likely to last, and more adaptable than a

black box. But some software requires enormous, sustained investment — medical devices, aerospace systems, specialized industrial tools — and the volunteer model does not always support that. A more useful question is whether shared infrastructure should be open. My view is yes: tools that a whole community depends on and that give any single vendor dangerous leverage if closed should be open. Git is a perfect example.

Is it ethical to build a profitable business on free community labor? GitHub was acquired by Microsoft for $7.5 billion. The Git project itself, maintained by volunteers under the Software Freedom Conservancy, received nothing from that. There is a credible defense: GitHub genuinely added value — the web interface, pull requests, Actions — and companies that add value can capture some of it. But the ethical obligation scales with the benefit. A startup using Git perhaps owes a good bug report. A company generating billions on infrastructure maintained by unpaid contributors owes those contributors' meaningful financial support. The industry does not provide this.

Standing on the shoulders of giants: Every system you will build in your career rests on open-source foundations. Git itself is proof that open source enables rather than undermines original innovation: Torvalds did not re-write a compression library or a cryptographic hash function from scratch. He built open tools and focused entirely on the new problem he was solving. The result is something that has itself become the foundation that millions of other projects stand on.

2. Linux Footprint

Installation: Debian/Ubuntu: Sudo apt install git. On Fedora/RHEL: sudo dnf install git. For the latest version, it can be compiled from sources using gcc, make, libssl-dev, libcurl4-openssl-dev, and zlib1g-dev — all of which are open-source tools.

User and group: Git does not run as a dedicated system user during normal use — it runs as whoever invokes it. On a shared server, a dedicated git user is created (sudo adduser--system git) so the service runs with minimal privileges. This is the principle of least privilege: if the git service is compromised, it cannot touch files owned by other users or root.

Service management: Git itself is not a persistent daemon. The git daemon command provides a read-only server that can be run as a system service (systems start git-daemon), but most teams use SSH instead — meaning the service being managed is OpenSSH, not Git directly. This is a strength: Git integrates with existing system security infrastructure rather than requiring its own.

Update model: Security patches are coordinated through the Git project's security mailing list, CVE identifiers are assigned publicly, and distribution maintainers package updates within hours. Running sudo apt upgrade git is all that is needed on most systems. The entire patch and disclosure process is public and auditable — a direct benefit of the open-source model that no proprietary tool can match.

3. The FOSS Ecosystem

Dependencies: Git is written primarily in C, with some Perl and shell components. Its core dependencies are OpenSSL/libcrypto for

hashing and HTTPS transport, libcurl for remote access, zlib for object compression, Perl for sub-commands like git-svn, and OpenSSH for authenticated push/pull. Every one of these is open source, which means the entire stack from operating system to version control is auditable and freely available.

What Git has enabled: GitHub (2008) added a web interface, pull requests, and social coding. on top of Git and was acquired for $7.5 billion. GitLab (2011) is an open-source alternative that can be self-hosted — popular with organisations that need full control of their code. Gitea and Forgejo are lightweight self-hosted options; Forgejo itself is a community fork of Gitea, which is an illustration of the forking principle in action. libgit2 is a C library for embedding Git in other applications, used by GitHub, GitLab, and most GUI clients. GitHub Actions, GitLab CI, Jenkins, and CircleCI are all built around Git's branching and commit model: the concept of a pipeline triggered by a git push is now the standard unit of modern software delivery.

Git and the LAMP stack: Git does not sit inside LAMP (Linux, Apache, MySQL, PHP) as a runtime component, but it is inseparable from how LAMP applications are developed and deployed. Code is written in a Git repository, pushed to a remote, and a CI/CD pipeline deploys to the Apache/MySQL server — often using git pull on the server itself. Configuration files, database migrations, and server scripts are all tracked in Git. It is the connective tissue of the entire development workflow.

Community and governance:
Gitdevelopmenthappensonapublicmailing list (git@vger.kernel.org)

entirely in plain-text email — deliberately transparent and low-technology. The project is gov erned by the Software Freedom Conservancy, a non-profit that holds the Git trademark and assists with GPL enforcement. Junio C. Hamano has been the maintainer since 2005. No company owns Git, and no acquisition can relicense or shut it down.

4. Shell Scripts

All five scripts were written and tested on a Linux system. Each is submitted as a sh file in Github repository.

4.1 Script1—System Identity Report.

```bash
# Script 1: System Identity Report
# Author: Tushar Chakraborty| Reg: 24BAI10842
# Course: Open Source Software (NGMC) | VITyarthi
# Purpose: Display a welcome screen summarising key Linux
#          system information and the OS license.
# Usage:   bash script1_system_identity.sh
# ============================================================

# ── Variables ──────────────────────────────────────────────
STUDENT_NAME="Tushar Chakraborty"
REG_NO="24BAI10842"
SOFTWARE_CHOICE="Git"

# ── Gather system information using command substitution ─────
KERNEL=$(uname -r)            # Kernel release version
ARCH=$(uname -m)              # Hardware architecture (x86_64 etc.)
USER_NAME=$(whoami)           # Name of the currently logged-in user
HOME_DIR=$HOME                # Home directory path from environment variable
HOSTNAME=$(hostname)          # Machine hostname
UPTIME=$(uptime -p)           # Human-readable uptime (e.g., "up 2 hours")
CURRENT_DATE=$(date '+%A, %d %B %Y')   # e.g., "Monday, 24 March 2026"
CURRENT_TIME=$(date '+%H:%M:%S')       # e.g., "14:30:00"

# ── Detect Linux distribution name ────────────────────────────
# /etc/os-release is the standard file for distro identification
# on modern systemd-based Linux distributions
if [ -f /etc/os-release ]; then
    # Source the file to load its variables, then extract PRETTY_NAME
    DISTRO=$(grep "^PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d '"')
else
    # Fallback if the file doesn't exist (very old systems)
    DISTRO="Unknown Linux Distribution"
fi

# ── Display formatted output ──────────────────────────────────
echo "============================================================"
echo "           OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT          "
echo "============================================================"
echo ""
echo "  Student : $STUDENT_NAME ($REG_NO)"
echo "  Project : $SOFTWARE_CHOICE — Open Source Software Audit"
echo ""
```

```
echo "  Project : $SOFTWARE_CHOICE — Open Source Software Audit
echo ""
echo "----------------------------------------------------------------"
echo "  SYSTEM INFORMATION"
echo "----------------------------------------------------------------"
echo "  Distribution : $DISTRO"
echo "  Kernel       : $KERNEL"
echo "  Architecture : $ARCH"
echo "  Hostname     : $HOSTNAME"
echo "  Logged in as : $USER_NAME"
echo "  Home Dir     : $HOME_DIR"
echo "  Uptime       : $UPTIME"
echo "  Date         : $CURRENT_DATE"
echo "  Time         : $CURRENT_TIME"
echo ""
echo "----------------------------------------------------------------"
echo "  LICENSE INFORMATION"
echo "----------------------------------------------------------------"
# Note: Most Linux distributions use the GPL v2 licensed kernel.
# Ubuntu and Debian ship with a mixture of free and non-free packages.
echo "  Linux Kernel License : GNU General Public License v2 (GPL v2)"
echo "  This means you have the freedom to:"
echo "    * Run the OS for any purpose"
echo "    * Study and modify the kernel source code"
echo "    * Redistribute copies freely"
echo "    * Share your modifications under GPL v2"
echo ""
echo "================================================================"
echo "  End of System Identity Report"
echo "=============
```

Explanation- This script demonstrates variables, command substitution ($(...)), a conditional file check (if [-f ...]), and formatted output with echo. Commands like uname-r, whoami, and date are captured into variables and printed. Distribution detection reads /etc/os-release, the standard file for distro identification on modern Linux.

4.2 Script 2 — FOSS Package Inspector

```
# Script 2: FOSS Package Inspector
# Author: Tushar Chakraborty | Reg: 24BAI10842
# Course: Open Source Software (NGMC) | VITyarthi
# Purpose: Check if a FOSS package is installed, display its
#          version and license, and print a philosophical note
#          about the package using a case statement.
# Usage:   bash script2_package_inspector.sh
# ==========================================================

# ── Target package — change this to test other packages ──────
PACKAGE="git"   # Primary package being audited in this project

# ── Helper function: check package on apt/dpkg systems ───────
check_package_apt() {
    # dpkg -l lists installed packages; grep -q = quiet (just checks exit code)
    # ^ii means the package is fully installed in dpkg state
    if dpkg -l "$1" 2>/dev/null | grep -q "^ii"; then
        echo "[INSTALLED] $1 is present on this system."
        echo ""
        echo "  Package details:"
        # dpkg -s gives detailed status; grep -E filters multiple fields
        dpkg -s "$1" 2>/dev/null | grep -E "^(Version|Maintainer|Homepage|Description)"
    else
        echo "[NOT FOUND] $1 is not installed via apt/dpkg."
        echo "  To install: sudo apt install $1"
    fi
}

# ── Helper function: check package on rpm systems ────────────
check_package_rpm() {
    # rpm -q queries if package is installed (exits 0 if yes, non-zero if not)
    if rpm -q "$1" &>/dev/null; then
        echo "[INSTALLED] $1 is present on this system."
        echo ""
        echo "  Package details:"
        # rpm -qi gives detailed info; grep filters version, license, summary
        rpm -qi "$1" | grep -E "^(Version|License|Summary|URL)"
    else
        echo "[NOT FOUND] $1 is not installed via rpm."
        echo "  To install: sudo dnf install $1"
    fi
}
```

```
check_package_rpm() {
    '1
}

# ── Detect which package manager is available ──────────────────
echo "================================================================"
echo "            FOSS PACKAGE INSPECTOR"
echo "================================================================"
echo ""
echo "  Checking for package: $PACKAGE"
echo "  ──────────────────────────────────"

# command -v returns the path to a command if it exists; used to detect tools
if command -v dpkg &>/dev/null; then
    # dpkg found: this is a Debian/Ubuntu based system
    check_package_apt "$PACKAGE"
elif command -v rpm &>/dev/null; then
    # rpm found: this is a Fedora/RHEL/CentOS based system
    check_package_rpm "$PACKAGE"
else
    echo "  Neither dpkg nor rpm found. Cannot query package database."
fi

echo ""

# ── Also show the git version directly if git command is available ──
if command -v git &>/dev/null; then
    echo "  Direct version check: $(git --version)"
fi

echo ""

# ── Case statement: print philosophy note for each package ────
# A case statement matches the value of PACKAGE against patterns.
# Each pattern ends with ;;  The * at the end is a wildcard default.
echo "================================================================"
echo "  OPEN SOURCE PHILOSOPHY NOTE"
echo "================================================================"
case $PACKAGE in
    git)
        # Git: the subject of this audit — GPL v2
        echo "  Git (GPL v2): Born from a licensing crisis in 2005, Git"
```

```
case $PACKAGE in
    git)
        # Git: the subject of this audit — GPL v2
        echo "  Git (GPL v2): Born from a licensing crisis in 2005, Git"
        echo "  proved that the best response to proprietary failure is to"
        echo "  build something free and better. Its distributed model is"
        echo "  a philosophical statement: history belongs to everyone."
        ;;
    httpd|apache2)
        # Apache HTTP Server — Apache 2.0 licence
        echo "  Apache (Apache 2.0): The web server that built the open"
        echo "  internet. Its permissive licence means anyone can build on"
        echo "  it commercially — a deliberate choice to maximise adoption."
        ;;
    mysql|mariadb)
        # MySQL/MariaDB — dual GPL v2 and commercial licence
        echo "  MySQL/MariaDB (GPL v2 / Commercial): MySQL's dual-licence"
        echo "  model is a lesson in open-source business. MariaDB forked"
        echo "  when Oracle acquired MySQL — community governance in action."
        ;;
    vlc)
        # VLC Media Player — LGPL/GPL
        echo "  VLC (LGPL/GPL): Started by students in Paris who wanted to"
        echo "  stream video on their university network. It plays anything."
        echo "  A reminder that student projects can change the world."
        ;;
    firefox)
        # Mozilla Firefox — MPL 2.0
        echo "  Firefox (MPL 2.0): A nonprofit browser fighting for an open"
        echo "  web against corporate monocultures. The fact that it exists"
        echo "  is itself an argument for the open-source model."
        ;;
    python3|python)
        # Python — PSF Licence
        echo "  Python (PSF Licence): Shaped entirely by community consensus"
        echo "  and the PEP process. The PSF licence is permissive, enabling"
        echo "  Python to become the most widely taught language on Earth."
        ;;
    *)
        # Default case: generic message for any other package
        echo "  $PACKAGE: Every FOSS project carries a philosophy in its"
```

```
    python3|python)
        # Python — PSF Licence
        echo "  Python (PSF Licence): Shaped entirely by community consensus"
        echo "  and the PEP process. The PSF licence is permissive, enabling"
        echo "  Python to become the most widely taught language on Earth."
        ;;
    *)
        # Default case: generic message for any other package
        echo "  $PACKAGE: Every FOSS project carries a philosophy in its"
        echo "  licence. Look up the SPDX identifier at spdx.org/licenses"
        echo "  to understand what freedoms its authors chose to grant."
        ;;
esac

echo ""
echo "================================================================"
echo "  End of FOSS Package Inspector"
echo "================================================================"
```

Explanation- This script uses if-then-else to detect the package manager and a nested if to check installation status. command-v tests whether a command exists. The case statement pattern-matches $PACKAGE against named strings; | lets multiple patterns

share one branch. grep-E uses extended regular expressions to filter multiple fields in one pipe

4.3 Script 3 — Disk and Permission Auditor

```bash
# Script 3: Disk and Permission Auditor
# Author: Tushar Chakraborty | Reg: 24BAI10842
# Course: Open Source Software (NGMC) | VITyarthi
# Purpose: Loop through key system directories and report
#          their disk usage, owner, group, and permissions.
#          Also check for Git's configuration directories.
# Usage:   bash script3_disk_permission_auditor.sh
# ==========================================================

# ── Array of directories to audit ──────────────────────────
# Arrays in Bash are defined with parentheses and space-separated values
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/var" "/usr/share")

# ── Display header ─────────────────────────────────────────
echo "============================================================"
echo "           DISK AND PERMISSION AUDITOR"
echo "           Auditing key Linux system directories"
echo "============================================================"
echo ""
# printf formats output into aligned columns using format specifiers
# %-20s = left-aligned string in 20-character wide field
printf "%-20s %-12s %-14s %-10s %s\n" "DIRECTORY" "SIZE" "PERMISSIONS" "OWNER" "GROUP"
echo "──────────────────────────────────────────────────────────────"

# ── For loop: iterate over each directory in the array ────────
# "${DIRS[@]}" expands all array elements; double quotes handle spaces
for DIR in "${DIRS[@]}"; do
    # [ -d "$DIR" ] checks if the path exists AND is a directory
    if [ -d "$DIR" ]; then
        # du -sh: disk usage summary in human-readable format
        # cut -f1: extract only the first tab-separated field (the size)
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        # ls -ld: long listing of the directory itself (not its contents)
        LS_OUTPUT=$(ls -ld "$DIR" 2>/dev/null)

        # awk extracts specific whitespace-separated fields from ls output
        # Field 1 = permissions, Field 3 = owner, Field 4 = group
        PERMS=$(echo "$LS_OUTPUT" | awk '{print $1}')
        OWNER=$(echo "$LS_OUTPUT" | awk '{print $3}')
        GROUP=$(echo "$LS_OUTPUT" | awk '{print $4}')
```

```
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then

        # Print formatted row
        printf "%-20s %-12s %-14s %-10s %s\n" "$DIR" "$SIZE" "$PERMS" "$OWNER" "$GROUP"
    else
        # If directory does not exist, print a clear message in same format
        printf "%-20s %s\n" "$DIR" "[ does not exist on this system ]"
    fi
done

echo "──────────────────────────────────────────────────────────────"
echo ""

# ── Section: Check Git-specific directories ────────────────────
echo "==============================================================="
echo "  GIT-SPECIFIC DIRECTORY CHECK"
echo "==============================================================="
echo ""

# Common locations where Git installs its files on Linux
GIT_DIRS=("/usr/share/git-core" "/usr/lib/git-core" "/etc/gitconfig")

for GDIR in "${GIT_DIRS[@]}"; do
    # -e checks existence of a file OR directory (more general than -d)
    if [ -e "$GDIR" ]; then
        LS_GIT=$(ls -ld "$GDIR" 2>/dev/null)
        GPERMS=$(echo "$LS_GIT" | awk '{print $1}')
        GOWNER=$(echo "$LS_GIT" | awk '{print $3}')
        echo "  Found: $GDIR"
        echo "    Permissions : $GPERMS"
        echo "    Owner       : $GOWNER"
    else
        echo "  Not found: $GDIR (Git may not be installed or path differs)"
    fi
    echo ""
done

# ── Section: Check current directory for a .git repository ──
echo "---------------------------------------------------------------"
echo "  Checking for .git repository in current directory..."
echo "---------------------------------------------------------------"
```

```
if [ -d ".git" ]; then
    echo ""
    echo "  A Git repository (.git/) exists in the current directory."
    # Get size and permissions of the .git directory itself
    GIT_SIZE=$(du -sh .git 2>/dev/null | cut -f1)
    GIT_PERMS=$(ls -ld .git | awk '{print $1}')
    echo "  .git/ size        : $GIT_SIZE"
    echo "  .git/ permissions : $GIT_PERMS"
    echo ""
    # Read the HEAD file to show the current branch reference
    if [ -f ".git/HEAD" ]; then
        echo "  Current HEAD: $(cat .git/HEAD)"
    fi
else
    echo ""
    echo "  No .git/ directory found here."
    echo "  Run from inside a Git repository to see repository-specific info."
fi

echo ""
echo "==============================================================="
echo "  End of Disk and Permission Auditor"
echo "==============================================================="
```

Explanation- The central construct is a loop over a Bash array. "${DIRS[@]}" expands all elements safely. du-sh reports disk usage in human-readable form; ls-ld lists directory metadata rather than contents; awk ’{print $N}’ extracts specific fields. printf with format specifiers produces column-aligned output. The second loop checks Git-specific paths using-e (file or directory existence).

4.4 Script 4 — Log File Analyzer

```bash
# ============================================================
# Script 4: Log File Analyzer
# Author: Tushar Chakraborty | Reg: 24BAI10842
# Course: Open Source Software (NGMC) | VITyarthi
# Purpose: Read a log file line by line, count occurrences
#          of a keyword, and print a summary with the last
#          5 matching lines. Accepts arguments from command line.
# Usage:   bash script4_log_analyzer.sh /var/log/syslog error
#          bash script4_log_analyzer.sh /var/log/auth.log warning
#          bash script4_log_analyzer.sh               (interactive retry)
# ============================================================

# ── Command-line arguments ──────────────────────────────
# $0 = script name, $1 = first argument, $2 = second argument
LOGFILE=$1

# Parameter expansion: ${2:-"error"} means "use $2, or 'error' if not set"
KEYWORD=${2:-"error"}

# ── Counters (integer variables) ────────────────────────
TOTAL_LINES=0    # Will count every line read from the file
MATCH_COUNT=0    # Will count lines that match the keyword

# ── Validate: ensure a log file argument was provided ────
if [ -z "$LOGFILE" ]; then
    # -z tests if a string is empty (zero length)
    echo "============================================================"
    echo "  ERROR: No log file specified."
    echo "  Usage: $0 /path/to/logfile [keyword]"
    echo "  Example: $0 /var/log/syslog error"
    echo "============================================================"
    exit 1   # Exit the script with a non-zero error code
fi

# ── do-while style retry loop ───────────────────────────
# Bash has no native do-while, but we simulate it with a while loop
# whose condition checks for an invalid file. We allow up to 2 retries.
MAX_RETRIES=2    # Maximum number of times to ask for a valid path
RETRY=0          # Counter starts at zero

# Loop continues as long as the file does NOT exist or is NOT readable
while [ ! -f "$LOGFILE" ] || [ ! -r "$LOGFILE" ]; do
```

```bash
# Loop continues as long as the file does NOT exist or is NOT readable
while [ ! -f "$LOGFILE" ] || [ ! -r "$LOGFILE" ]; do
    RETRY=$((RETRY + 1))   # Arithmetic: increment retry counter
    echo ""
    echo "  [WARNING] File not found or not readable: $LOGFILE"

    # If we have hit the retry limit, give up and exit
    if [ $RETRY -ge $MAX_RETRIES ]; then
        echo "  [ERROR] Maximum retries exceeded. Exiting."
        echo ""
        echo "  TIP: Try using /var/log/syslog or creating a test file:"
        echo "       echo 'test error line' > /tmp/test.log"
        echo "       bash $0 /tmp/test.log error"
        exit 1
    fi

    # Prompt the user for a new path
    echo "  Please enter a valid log file path (attempt $RETRY of $MAX_RETRIES):"
    read -r LOGFILE
done

# ── Display header ──────────────────────────────────────────
echo ""
echo "================================================================"
echo "  LOG FILE ANALYZER"
echo "  File    : $LOGFILE"
echo "  Keyword : '$KEYWORD' (case-insensitive search)"
echo "================================================================"
echo ""

# ── Main loop: read file line by line ───────────────────────
# IFS= prevents leading/trailing whitespace stripping
# read -r prevents backslash interpretation
# < "$LOGFILE" redirects the file into stdin for the while loop
while IFS= read -r LINE; do

    # Increment total line counter using arithmetic expansion
    TOTAL_LINES=$((TOTAL_LINES + 1))

    # Check if this line contains the keyword
    # grep -i = case-insensitive, -q = quiet (no output, just return code)
```

```
while IFS= read -r LINE; do
done < "$LOGFILE"

# ── Print summary results ──────────────────────────────────
echo "  SUMMARY"
echo "  ──────────────────────────────────────────────────────────────"
echo "  Total lines in file      : $TOTAL_LINES"
echo "  Lines matching '$KEYWORD' : $MATCH_COUNT"

# Calculate match percentage using integer arithmetic
# Bash only does integers, so multiply by 100 BEFORE dividing
if [ "$TOTAL_LINES" -gt 0 ]; then
    PERCENT=$(( (MATCH_COUNT * 100) / TOTAL_LINES ))
    echo "  Match percentage         : ${PERCENT}%"
else
    echo "  The file appears to be empty."
fi

echo ""
echo "  LAST 5 MATCHING LINES"
echo "  ──────────────────────────────────────────────────────────────"

# grep -i searches case-insensitively; pipe to tail -5 to get last 5 results
# 2>/dev/null suppresses error messages if the file disappears mid-run
LAST_MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" 2>/dev/null | tail -5)

if [ -z "$LAST_MATCHES" ]; then
    # -z checks if string is empty: no matches found
    echo "  (No lines matching '$KEYWORD' were found in the file)"
else
    # Print each matching line with a >> prefix for visibility
    echo "$LAST_MATCHES" | while IFS= read -r MATCH_LINE; do
        echo "  >> $MATCH_LINE"
    done
fi

echo ""
echo "================================================================"
echo "  End of Log File Analyzer"
echo "================================================================"
```

Explanation: The key construct is while IFS=read-r LINE—the correct Bash pattern for reading a file line by line. The file is fed via input direction(<"$LOGFILE"), not via cat. Command-linear arguments($1,$2) and parameter expansion for defaults(${2:-"error"}) are demonstrated. Ado-while style retry looper-prompts for a valid file path. Integer arithmetic uses$((...)).

4.5 Script5—Open-Source Manifesto Generator

shcode > $ script5.sh

```bash
# ============================================================
# Script 5: The Open Source Manifesto Generator
# Author: Tushar Chakraborty | Reg: 24BAI10842
# Course: Open Source Software (NGMC) | VITyarthi
# Purpose: Interactively ask the user three questions, then
#          compose a personalised open-source philosophy
#          statement and save it to a .txt file.
# Usage:   bash script5_manifesto_generator.sh
# ============================================================

# ── Alias concept (demonstrated in comments) ──────────────
# In an interactive shell session, you can define shortcuts like:
#   alias greet='echo "Welcome to the Open Source Manifesto Generator"'
# Aliases are command shortcuts defined with the 'alias' keyword.
# They are stored in ~/.bashrc for persistence across sessions.
# Example: alias ll='ls -la' makes 'll' run the full listing command.
# Note: Aliases are NOT available inside non-interactive scripts by default,
# so we demonstrate the concept here in comments rather than using one.

# ── Welcome banner ────────────────────────────────────────
echo "================================================================"
echo "       THE OPEN SOURCE MANIFESTO GENERATOR"
echo "       A VITyarthi OSS Project Tool"
echo "       Author: Tushar Chakraborty | 24BAI10842"
echo "================================================================"
echo ""
echo "  Answer three questions to generate your personal open-source"
echo "  philosophy statement. Your answers will be composed into a"
echo "  paragraph and saved to a text file."
echo ""
echo "----------------------------------------------------------------"
echo ""

# ── Read user input interactively ─────────────────────────
# read -r: reads a line from stdin; -r prevents backslash escaping
# read -p "prompt": displays an inline prompt before reading
# The answer is stored in the variable after -p "..."

read -r -p "  Q1. Name one open-source tool you use every day: " TOOL

# Input validation: loop until the user gives a non-empty answer
# -z tests for an empty (zero-length) string
```

```
read -r -p "  Q1. Name one open-source tool you use every day: " TOOL

# Input validation: loop until the user gives a non-empty answer
# -z tests for an empty (zero-length) string
while [ -z "$TOOL" ]; do
    echo "  Please type something — this question cannot be skipped."
    read -r -p "  Q1. Name one open-source tool you use every day: " TOOL
done

echo ""
read -r -p "  Q2. In one word, what does 'freedom' in software mean to you? " FREEDOM

while [ -z "$FREEDOM" ]; do
    echo "  Please enter at least one word."
    read -r -p "  Q2. In one word, what does 'freedom' mean to you? " FREEDOM
done

echo ""
read -r -p "  Q3. Name one thing you would build and share freely if you could: " BUILD

while [ -z "$BUILD" ]; do
    echo "  Please type your answer — even a small idea counts."
    read -r -p "  Q3. Name one thing you would build and share freely: " BUILD
done

echo ""
echo "----------------------------------------------------------------"
echo "  Composing your manifesto..."
echo "----------------------------------------------------------------"
echo ""

# ── Prepare variables ──────────────────────────────────────────
# date with a format string produces a readable timestamp
DATE=$(date '+%d %B %Y')     # e.g., 24 March 2026
TIME=$(date '+%H:%M')        # e.g., 14:30

# The output filename includes the current username for uniqueness
# String concatenation in Bash: just place variables adjacent to text
OUTPUT="manifesto_$(whoami).txt"

# ── Write the manifesto to a file ──────────────────────────────
```

```
# > (redirect) creates/overwrites the file with the first line
# >> (append redirect) adds subsequent lines without erasing previous ones

# Write the file header
echo "================================================================" > "$OUTPUT"
echo "  MY OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo "  By: Tushar Chakraborty (24BAI10842)" >> "$OUTPUT"
echo "  Generated: $DATE at $TIME" >> "$OUTPUT"
echo "================================================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"

# Write paragraph 1 — using $TOOL (string interpolation inside double quotes)
echo "  Every day I rely on $TOOL — a tool I did not have to buy," >> "$OUTPUT"
echo "  did not have to beg permission to use, and can inspect down" >> "$OUTPUT"
echo "  to its last line of code if I choose to. This is not a small" >> "$OUTPUT"
echo "  thing. It is the direct result of people choosing to build" >> "$OUTPUT"
echo "  in the open and share their work freely." >> "$OUTPUT"
echo "" >> "$OUTPUT"

# Write paragraph 2 — using $FREEDOM
echo "  To me, freedom in software means $FREEDOM. Not in an abstract" >> "$OUTPUT"
echo "  way, but in the practical sense: freedom to run, to study," >> "$OUTPUT"
echo "  to modify, and to share. The four freedoms that the Free" >> "$OUTPUT"
echo "  Software Foundation articulates are not idealism. They are" >> "$OUTPUT"
echo "  the conditions under which knowledge compounds rather than" >> "$OUTPUT"
echo "  stagnates. Every open-source tool that exists is evidence" >> "$OUTPUT"
echo "  of what becomes possible when those conditions are met." >> "$OUTPUT"
echo "" >> "$OUTPUT"

# Write paragraph 3 — using $BUILD
echo "  If I could build one thing and share it freely, it would be" >> "$OUTPUT"
echo "  $BUILD. I would license it under GPL v2 — not because" >> "$OUTPUT"
echo "  it would make me famous, but because it is the surest way" >> "$OUTPUT"
echo "  to ensure that whatever I build remains available, forever," >> "$OUTPUT"
echo "  to everyone who might find it useful. Standing on the" >> "$OUTPUT"
echo "  shoulders of giants means, eventually, being someone else's" >> "$OUTPUT"
echo "  giant. This is my commitment to that chain." >> "$OUTPUT"
echo "" >> "$OUTPUT"

# Write footer
echo "----------------------------------------------------------------" >> "$OUTPUT"
echo "  Course: Open Source Software (NGMC) | VITyarthi 2026" >> "$OUTPUT"
```

```
# Write footer
echo "----------------------------------------------------------------" >> "$OUTPUT"
echo "  Course: Open Source Software (NGMC) | VITyarthi 2026" >> "$OUTPUT"
echo "================================================================" >> "$OUTPUT"

# ── Confirm save and display the manifesto ──────────────────────
echo "  Manifesto successfully saved to: $OUTPUT"
echo ""
echo "================================================================"
echo "  YOUR GENERATED MANIFESTO"
echo "================================================================"
echo ""

# cat reads the file and prints its contents to the terminal
cat "$OUTPUT"

echo ""
echo "================================================================"
echo "  End of Manifesto Generator"
echo "================================================================"
```

Explanation: Script 5 uses read-r-p for interactive input. Input validation is used while [-z "$VAR"] to re-prompt until the user answers. String concatenation in Bash needs no operator — placing a variable inside a double-quoted string interpolates it. File writing uses > to create the file and » to append subsequent lines. The alias concept is explained in the header comments. cat displays the saved file on screen after writing.

5. Conclusion:

Git started as a crisis response and became infrastructure that modern software development cannot exist without. What this audit has tried to show is that this outcome was not accidental —it was a direct result of the license choice and the design philosophy. Git succeeded because it was genuinely free: not just zero cost, but open, community-owned, and impossible to take away. The BitKeeper story is the clearest possible proof of why that matters. A tool that tens of thousands of people depended on disappeared overnight because one company changed its mind. That cannot happen to Git. The Linux footprint analysis showed how cleanly Git integrates with the operating system — no bloated service, no special permissions, just files and a well-designed command-line interface built on standard Unix tools. The ecosystem section showed what becomes possible when a well-designed tool is truly open: hundreds of platforms, clients, and workflows built on top of it without anyone asking permission. The comparison with Perforce was honest: there are specific use cases where proprietary software has real advantages. But for the vast majority of software projects,

Git is the right answer, and the fact that it is free and community-governed only strengthens that.

The five shell scripts in this project are more than assessment tasks. Writing and running them meant working directly in the Linux environment that Git was built to serve — checking permissions, reading directories, parsing log files, and writing output to files. That practical familiarity with the tool's underneath is the foundation that the rest of a software career is built on.

References

1. GNU Project. Free Software Definition. https://www.gnu.org/philosophy/free-sw. html

2. Open-Source Initiative. The Open-Source Definition. https://opensource.org/osd

3. Raymond- E.S. Cathedral and the Bazaar. http://catb.org/~esr/writings/cathedral-bazaar/

4. Git Source Repository and COPYING file. https://github.com/git/git

5. Software Freedom Conservancy. https://sfconservancy.org/

6. Git Reference Manual. https://git-scm.com/docs

7. SPDX Licence List. https://spdx.org/licenses/

8. Shotts, W. The Linux Command Line. https://linuxcommand.org

9. GNU Bash Manual. https://www.gnu.org/software/bash/manual/

10. Stack Overflow Developer Survey 2023. https://survey.stackoverflow.co/2023